

SpGEMM(稀疏矩阵乘法)研究文献综述

本文档对 [ref/](#) 目录下 8 篇 SpGEMM 论文进行阅读总结与深度对比,并补充主流稀疏矩阵存储格式作为背景知识。

论文覆盖两条技术路线:**GPU 软件算法**(4 篇)与**专用硬件加速器**(4 篇)。

0. 文章概览

#	论文	类型	平台	核心思路
1	Adaptive (AC-SpGEMM) · PPoPP'19	GPU 软件	GPU	自适应 chunk + 多轮片上 ESC, 保证 bit-stable
2	Efficient GPU for Irregular Data (Liu & Vinter) · IPDPS'14	GPU 软件	GPU	按行长分 5 桶 38 组,差异化 kernel + 混合预分配
3	spECK · 2017 (Pascal)	GPU 软件	GPU	哈希表放共享内存 + 行分组,高性能且省内存
4	HSMU-SpGEMM · 2025	GPU 软件	GPU	排序列数组 + 二分查找替代哈希,零冲突
5	HIRAC (Lehigh/Qualcomm)	硬件加速器	FPGA/ASIC	SorPack 排序打包 + 层级本地合并,去掉复杂互连
6	MatRaptor · HPEC'20 (UMBC)	硬件加速器	FPGA	Row-wise product 数据流,稀疏-稀疏
7	Sparm · ASAP'24 (NUDT)	硬件加速器	ASIC 模拟	多数据流 + 正则化合并(取代 Flexagon MRN)
8	SpAda · ASPLOS'23	硬件加速器	ASIC 模拟	窗口自适应数据流,运行时动态切换 IP/OP/行式

1. 背景知识:主流稀疏矩阵存储格式

稀疏矩阵的"存储格式"决定了数据在内存中的布局,直接决定访存模式、向量化效率、以及 SpGEMM 算法的选择。下面按"通用基础格式 → 结构化/分块格式 → 高维与位图格式"梳理。

1.1 基础通用格式

COO(Coordinate Format,坐标格式)

- **结构:**三个等长数组 `row[]`、`col[]`、`val[]`,每个非零元存一个 (行, 列, 值) 三元组。
- **优点:**构造简单、增量插入容易、易做转置;适合矩阵组装阶段。
- **缺点:**访存不合并;SpMV 效率低。
- **典型用途:**矩阵的初始构建、COO↔CSR 转换、溢出元素的兜底存储。

CSR(Compressed Sparse Row,压缩稀疏行) ★

- **结构:** `row_ptr[n+1]` (每行起始偏移) + `col_idx[nnz]` + `val[nnz]`。
- **优点:** 按行压缩,行切片高效;SpMV 的**事实标准**;列索引行内有序时利于合并/去重。
- **缺点:** 按列访问(如取一列)效率差;行间负载不均衡。
- **典型用途:** 本文绝大多数 SpGEMM 论文的输入/输出格式(AC-SpGEMM、spECK、Liu&Vinter、HSMU 等均基于 CSR)。

CSC(Compressed Sparse Column,压缩稀疏列)

- 与 CSR 完全对称,只是以列为主。
- **SpGEMM 中的角色:** 计算 $C = A \times B$ 时, B 按列访问更自然,因此不少实现会把 B 转成 CSC,或同时维护 CSR/CSC 双格式。

DIA(Diagonal Storage,对角线存储)

- 按对角线存储,适合**带状矩阵**(非零元集中在对角线附近,如 Poisson/PDE 离散矩阵)。
- SpMV 高效但对一般稀疏矩阵浪费严重。

1.2 规则/向量化友好格式

ELL(ELLpack)

- **结构:** 把每行对齐到相同的 K 个非零(`col_idx[rows][K]`、`val[rows][K]`),不足的用零 padding。
- **优点:** 访存完全规则,**向量化/GPU 友好**,SpMV 性能高。
- **缺点:** 若各行非零数差异大,padding 浪费严重(取决于最长行)。

HYB(Hybrid = ELL + COO)

- **结构:** 大部分非零放 ELL(取每行前 K 个,K 通常取某分位数),溢出的"长尾"放 COO。
- **优点:** 兼顾向量化和负载均衡;早期 **cuSPARSE 推荐的 SpMV 格式**。
- **缺点:** 需选择合适的 K;两套数据结构。

1.3 分块与结构化格式(DNN / 结构化剪枝常用)

BCSR / BSR(Block CSR)

- 把矩阵分成固定大小(如 2×2 、 4×4)的小块,以块为单位做 CSR 式压缩。
- **用途:** **结构化剪枝 DNN**(块稀疏)、分块稠密子矩阵加速(Tensor Core 友好)。

ELLPACK / SELLPACK

- ELL 的分块/分组变体,按行分片允许不同 K,缓解 padding 浪费。

1.4 高维与位图格式(图分析 / 本文论文特有)

CSF(Compressed Sparse Fiber)

- 树状层级压缩,原为高维稀疏张量设计,也用于图。

- 压缩率高,但随机访问与修改成本高。

Bitmap / Bitmask(位图) ★ 本综述多篇论文的关键

- 用一个 bit 标记某位置是否非零(1 bit/元素),配合数组存值。
- 在本文中的应用:
 - **HSMU-SpGEMM**:symbolic 阶段用 **mask matrix** + 按位 OR 生成 mask C,再用 `popc()`(统计置位)、`ffsll()`(找首置位)提取信息;大规模时用**压缩 mask**(三数组 `tilePtr` / `tileColIdx` / `tileMask`,每 64 位一单元)。
 - **Sparm**:对 A 的行、B 的列分别维护 **bitmap**,离线做按位 AND 求交集,预计算 **bitindex** 表指导运行时合并调度。

格式选择的经验法则:

- 通用 SpGEMM / SpMV、行访问为主 → **CSR**
- GPU 向量化 SpMV、规则稀疏 → **ELL / HYB**
- 结构化剪枝 DNN → **BCSR**
- 只需知道"有没有非零"、做交集/并集运算 → **bitmap**(位运算极快,本文 HSMU/Sparm 的性能来源之一)

2. SpGEMM 的共同核心难点

SpGEMM ($C = A \cdot B$, 稀疏 $\times$ 稀疏) 之所以比稠密 GEMM、SpMV 都难,本质是三个"不可预测":

1. **输出非零元个数不可预测** → 无法预先精确分配内存,只能两遍式(先计数再计算)或预分配上界(浪费)。
2. **中间乘积 ($a_{ik} \cdot b_{kj}$) 数量巨大且需合并去重** → 合并(merge/reduce)阶段往往是真正瓶颈,远贵于乘法本身。
3. **稀疏结构高度不规则** → 负载不均衡,行长差异可达数千倍。

加之应用两极分化:

- **图分析**矩阵极庞大,非零率低至 10^{-6} ;
- **剪枝 DNN** 矩阵小但稀疏度仅 10%–90%。

没有单一数据流/算法能通吃所有稀疏模式 —— 这是后 4 篇(尤其 SpAda、Sparm)反复强调的结论,也是"自适应"成为研究热点的原因。

3. GPU 软件方案(论文 1–4)

3.1 各篇要点

① AC-SpGEMM(Adaptive, PPOPP'19)

- **四阶段**:全局负载均衡 → 自适应分块 ESC(AC-ESC) → Chunk 合并 → 输出拷贝。
- **选 ESC 的理由**:稳定排序 → **bit-stable(确定性)**结果,规避哈希的调度依赖不确定性。
- **核心创新**:
 - 仅按 A 的非零元均匀切分给 block,跳过中间乘积预枚举;
 - **多轮片上 ESC**:无视行边界多次本地迭代,仅行完成或内存满才写全局显存;

- 动态位宽缩减的 Radix 排序(本地行 id 重映射);
- 单次 prefix scan 同时完成压缩/计数/写出(32 位状态字编码);
- restart 机制(chunk pool 不够时换出续算)。
- **结果:**高度稀疏矩阵上平均比 cuSPARSE/bhSparse/RMerge/nsparse/Kokkos 快 **3×-4×**;需 bit-stable 时几乎全场景最优,最佳案例快 **20×**。
- **弱点:**中间乘积极多(压缩因子高)的矩阵输给 nsparse 等哈希方案。

② Liu & Vinter(Efficient GPU for Irregular Data, IPDPS'14)

- **四阶段:**算上界 → 分桶 → 计算 → 整理。
- **核心创新——按行长精细分桶:**5 个桶组、**38 个桶**:
 - 长度 0/1:平凡;
 - 长度 2-32(31 个桶):一行一线程,每桶独立 kernel → 天然负载均衡,scratchpad 上 **heap 方法**;
 - 长度 33-512(4 个桶):一组线程/行,**bitonic ESC**;
 - 长度 >512:大行,**gather/compress + GPU merge path**。
- **混合内存预分配:**短行用上界法,长行用渐进法(**nnz=256** 起步扩展),兼顾内存与效率。
- **意义:**首个明确针对"任意不规则稀疏结构"都高效的 GPU SpGEMM(此前 GPU 方法只对较规则矩阵好)。

③ spECK(High-Performance & Memory-Saving, Pascal, 2017)

- **两阶段:**计数 → 计算(CSR 输入输出)。
- **核心创新:**
 - **哈希表放共享内存**(关键),冲突用线性探测 + **atomicCAS**;
 - **按行分组**(7 组,Group 0-6),每组配不同哈希表大小/线程块大小,组越往下尺寸减半以提高 occupancy;
 - 仅最长行(Group 0)回退全局内存哈希表;
 - 共享内存放不下时记录行号,二次用全局内存处理。
- **结果:**比 cuSPARSE 几何平均 **1.99×**、比 CUSP(ESC)**4.31×**;工作内存平均省 **4.34×**(最高 40×)——"Memory-Saving"标题的来源。

④ HSMU-SpGEMM(2025)

- **针对痛点:**基于哈希累加器的两大缺陷——**哈希冲突**(Nsparse 冲突比 400%+)与**共享内存浪费**(OpSparse 利用率仅 35.35%)。
- **核心创新——排序列数组 + 二分查找替代哈希:**
 - symbolic 阶段就提取 C 的 NNZ,生成**已排序的 Ccol**;
 - numeric 阶段用 **findInSorted()**(O(log N) 二分查找)定位,再 **atomicAdd** 累加;
 - **零哈希冲突,共享内存利用率 71.69%**;
 - 混合累加器:NNZ ≤ 4096 用二分查找累加器,否则用稠密累加器(借鉴 spECK);
 - 大规模 symbolic 用**压缩 mask 格式**。
- **结果:**RTX 3090 Ti 上几何平均 **3.19×**(最高 131.64×);极大规模矩阵 Nsparse/cuSPARSE OOM 而 HSMU 仍可跑。

3.2 🔍 深度对比:GPU 四篇的"合并策略 × 负载均衡 × 内存"三角

维度	AC-SpGEMM (2019)	Liu&Vinter (2014)	spECK (2017)	HSMU (2025)
中间结果合并方式	ESC(展开-排序-压缩)	heap / bitonic sort / merge	哈希表(线性探测 + atomicCAS)	排序数组 + 二分查找
确定性(bit-stable)	✅ 强保证	✅ (排序确定)	❌ (哈希,调度依赖)	✅ (排序确定)
负载均衡粒度	按 A 非零元均匀切分 block	按行长分 38 桶(最细)	按中间乘积数分 7 组	按行,一 warp 处理一个 C 元素
工作数据存放	片上 scratchpad(多轮 ESC)	scratchpad(短行)/全局(长行)	共享内存哈希表(关键)	共享内存排序列数组
内存节省	一般(保守估计过度分配)	混合预分配(短上界/长渐进)	省 4.34×(标题卖点)	高利用率,大矩阵不 OOM
最强场景	高度稀疏 + 需确定性	极不规则矩阵	行长适中,省内存	大规模、行较长
最弱场景	中间乘积极多(压缩因子高)	行极长时退化为渐进法	超长行回退全局内存	低端 GPU(共享内存小)优势缩小

演进脉络(关键结论):

GPU 阵营的演进是 "合并策略"的单线进化: ESC(确定性但费内存)→ 哈希(快但冲突/不确定)→ 排序数组 + 二分查找(快、确定、零冲突)。

HSMU(2025)之所以成为目前最优,正因为它同时拿到了哈希的速度和 ESC 的确定性,并用排序数组消除了哈希的固有"冲突 ↔ 利用率"两难。负载均衡则一路从"按行长分桶"(Liu&Vinter/spECK)走向"按 A 非零元均匀切分"(AC-SpGEMM)。

4. 硬件加速器方案(论文 5–8)

4.1 各篇要点

⑤ HIRAC(Lehigh/Qualcomm)

- 目标:DNN 推理的中等稀疏 SpGEMM(激活稀疏 50–98%、权重稀疏 10–90%),内积数据流。
- 软件 SorPack:分区 → 按每行/列非零数排序 → 单遍打包 → 删空行。目的是让需合并的部分和在时空上靠近,最小化"列分裂距离"。
- 硬件层级:PE 子阵列(含 Same Cycle Merger + PS Buffer)→ 极简单向互连(刻意简化)→ 分块 SRAM。
- 结果:比 SIGMA 平均 3.2× 加速、面积 –9.5%、功耗 –32%;端到端 DNN 比 TPU 快 8.2×(并行 SorPack)。

⑥ MatRaptor(UMBC, HPEC'20)

- 目标:稀疏-稀疏 SpMM,基于 Row-wise product 数据流。
- 思路:A 行分布到 PE,与 B 对应行做"逐行乘加",等价于行外积求和;分布式产生 + 树状归约;按 NNZ 切分 A 行做负载均衡。
- 特点:相比 inner-product 无需沿 k 全归并,相比 outer-product 中间结果更少且按行局部化。

- **结果:**比 CPU MKL 数倍至数十倍;能效(GFLOP/s/W)明显优于 GPU。
- **局限:**B 接近稠密时 row-wise 笛卡尔积中间项爆炸,优势消失。

⑦ Sparm(NUDT, ASAP'24)

- **目标:**支持 **IP / OP / Gustavson 三种数据流**,针对 Flexagon 的 MRN(坐标匹配合并)在高度稀疏时阻塞严重的缺陷。
- **核心创新——正则化合并(Regularized Merging):**
 - **离线:**A 行 bitmap 与 B 列 bitmap 按位 AND → 预计算 **bitindex 表**;离线确定最优数据流;
 - **运行时:**乘法结果先进乘法器后小 FIFO,按列坐标分配 **K_ID**,Merge Manager 按 bitindex 表只调度相同 K_ID 的 FIFO 弹出 → **移除比较器**,合并树比 MRN 还小;
 - **RC Prefetcher + Look-Ahead FIFO:**Gustavson 下提前预取 B 行,提升 StrCache 命中率。
- **结果:**高度稀疏(>95%)时比 SIGMA 最高 **223x**;9 组真实负载平均比 Flexagon **1.35x**。

⑧ SpAda(ASPLOS'23)

- **核心洞察:**IP / OP / Row-wise 三种数据流其实是同一**"窗口"模板**在不同 **height** 下的特例:
 - height=1 + 单通道组 → 类行式;
 - height=2 + 2 通道 → 行式;
 - height 更大 → 外积风格。
- **硬件:**PE 含多通道,可动态划分为**"合并组"**;专用排序归约硬件实现快速重配置;**调度器运行时检测稀疏模式并自适应切换** WA 模式(用前几行 IPC/利用率指导后续行)。
- **结果:**比 SIGMA 约 38x、比 SpArch 1.44x、比 GAMMA 1.46x;**鲁棒性优势**——无论负载偏向哪种数据流,都能匹配该专用加速器而不退化。

4.2 🔍 深度对比:硬件四篇的"数据流 × 合并网络 × 自适应"

维度	HIRAC	MatRaptor	Sparm	SpAda
支持数据流	仅内积(IP)	仅行式(row-wise product)	IP/OP/Gustavson 三种	统一窗口模板,运行时切换
部分和合并机制	本地 Same Cycle Merger + PS Buffer	行局部化 + 树状归约	bitindex 调度 FIFO(无比较器)	专用排序归约树 + 累加器
简化互连的关键手段	SorPack 排序 让部分和聚拢	row-wise 把合并按行局部化	离线 bitmap 预计算	重配置通道组
自适应能力	参数 P / 子阵列大小调参	无(固定数据流)	离线选数据流(运行时自适应留作未来)	运行时动态自适应(最强)
目标场景	中等稀疏 DNN	稀疏-稀疏图/GNN	高度稀疏(>95%)	混合稀疏模式负载
典型加速	vs SIGMA 3.2x、功耗 -32%	vs MKL 5-40x	vs SIGMA 最高 223x	vs SpArch 1.44x

核心结论:

硬件阵营的关键战场是**"部分和怎么合并"**。两条路线殊途同归:

- **HIRAC、Sparm:**用**算法预处理重塑数据布局**(排序打包 / bitmap 预计算),让需要合并的部分和**自然聚到一起**,从而用最简单的本地合并硬件解决,砍掉 SIGMA/Flexagon 那种复杂互连网络。
- **SpAda:**承认"没有万能数据流",用**统一可重配置模板 + 运行时自适应**通吃各种稀疏模式。

这指向同一信条:**软硬件协同 > 纯硬件堆砌**。与其加更多 PE、更复杂 NoC,不如在算法/调度层消除合并的散乱。

5. 全景横向对比

维度	GPU 软件阵营	硬件加速器阵营
核心瓶颈	片上(共享内存)合并 + 负载均衡	部分和合并网络 + 数据流选择
主流合并手段	ESC / 哈希 / 排序数组+二分	本地合并器 + bitindex 调度 + 排序归约树
自适应趋势	AC-SpGEMM 自适应 chunk	SpAda 运行时切换数据流、Sparm 多数据流
优势	通用、可移植、生态成熟	能效极高、定制化性能
劣势	受限于 GPU 内存层级与带宽	灵活性差、研发/流片成本高
最适合	通用科学计算、大规模图	DNN 推理、能效敏感的边缘部署

6. 三条贯穿全文的核心洞察

1. **"合并"是 SpGEMM 的第一性难题,不是"乘法"**。无论 GPU(共享内存里的 chunk/hash/排序数组)还是 ASIC(合并网络),谁降低合并的访存 / 比较 / 阻塞开销,谁就领先。8 篇论文无一例外都在啃这块骨头。
2. **自适应 / 数据流选择是明确趋势**。固定单一算法或数据流必然在某些稀疏模式上退化 → AC-SpGEMM 的自适应 chunk、SpAda 的窗口自适应、Sparm 的多数据流,都指向"按输入形态动态选择策略"。Sparm 明确把"运行时自动选最优数据流"列为未来工作。
3. **软硬件协同优于纯硬件堆砌**。HIRAC(SorPack 排序让部分和聚拢)、Sparm(离线 bitmap 预计算)都证明:**用算法预处理重塑数据布局,可以大幅简化硬件**——这是比"加更多 PE / 更复杂 NoC"更划算的方向。

附录:存储格式 ↔ 论文对应速查

论文	主要用到的格式
AC-SpGEMM	CSR(输入输出),chunk + per-row 链表(中间)
Liu & Vinter	CSR,scratchpad 数组,分桶
spECK	CSR,共享内存哈希表
HSMU	CSR, mask matrix + 压缩 mask(tilePtr/tileColIdx/tileMask) ,排序列数组
HIRAC	打包(packed)矩阵,stationary/streaming
MatRaptor	CSR-like + 行指针表(row-wise)
Sparm	bitmap + bitindex 表 ,StaFIFO/StrCache/PSRAM

论文	主要用到的格式
SpAda	窗口化布局,通道组